DATA ENGINEERING

AI · ML · DS / REVISION NOTES

YOUTUBE · INTELLIPAAT

PySpark — Part 2

A full PySpark walkthrough, from setup to DataFrame operations

24
PAGES

Compiled October 2026
github.com/ankitgupta1729/AI-ML-DS

Formatted from Notes2.txt (content preserved)

> Source: Intellipaat (https://www.youtube.com/watch?v=QUiAc3rWtMA)

> (1) Spark Stack contains:

- A. Spark Core -- The underlying engine
- B. Spark SQL -- For SQL and unstructured data processing
- C. Spark Streaming -- Stream processing of the live data streams
- D. MLlib -- Machine Learning algorithms
- E. GraphX -- Graph processing

> (2) The spark core forms the vital component of spark and it is the base of Spark engine.

It contains the basic functionalities of Spark, including task scheduling (like FCFS,round robin), memory management, fault recovery etc.

> (3) The main programming abstraction for Spark core is RDD which is its basic data abstraction.

> (4) Spark Core provides many APIs for processing structured and unstructured data.

> (5) Spark SQL provides integrated APIs to work with structured data

> (6) It has a great set of APIs that allows querying data via SQL commands

> (7) It supports various data sources like CSV, JSON, Parquet, Hive, Cassandra etc.

> (8) Spark Streaming enables the processing of live streams of data

> (9) It divides the live input data streams into batches

> (10) For instance, logs being generated from an app server or live tweets from Twitter can be processed in near real time

> `using Spark Streaming APIs.`

> (11) Spark MLlib provides in-built libraries to implement ML algorithms like classification, regression, clustering, collaborative filtering

> (12) Spark MLlib also provides various techniques for data preprocessing

---

> (13) GraphX is a library for manipulating graphs. It provides analysis and graph computation for big data.

> (14) GraphX comes with a variety of graph algorithms and graph computations

> (15) GraphX can be used for the disaster detection system, page ranking, financial fraud detection etc.

> (16) Programming Languages for spark: Python, Scala, R, Java

Libraries: Spark SQL, MLlib, GraphX, Streaming

Engine: Spark Core

Cluster Management Tools: Hadoop Yarn, Apache Mesos, Spark Scheduler

Storage tools: HDFS (Hadoop File System), Standalone Node, Cloud, RDBMS/NoSQL

> (17) Spark Architecture:

we need spark to understand data. So, our driver program (or drivers) consists of "SparkContext" which is used to create
RDDs. For every single instance of spark running or spark core running, we need one

SparkContext. So, one SparkContext for
every single program.

This driver program is connected with "Cluster Management" tools which is required to pull data from various nodes.

Now this Cluster Manager is connected with "Worker Node" which are the endpoint of our spark architecture where data
transformed into information where we getting the task done. It is having "Executor" for Cache and Task in various working nodes.

> (18) Apache Spark provides a well-defined layered architecture.

> (19) All Spark components and layers are loosely coupled.

> (20) A driver program runs on the master node of the Spark cluster

> (21) It schedules the job execution and negotiates with the cluster manager

> (22) It also translates RDDs into the execution graphs. (consider RDDs as arrays)

> (23) The driver program can split graphs into multiple stages

> (24) The executor is a distributed agent responsible for the execution of tasks

> (25) Every Spark application has its own executor process

> (26) An executor performs all data processing

> (27) It reads data from and writes data to external sources

> (28) It interacts with storage systems

> (29) Cluster Manager is an external service responsible for acquiring resources on the Spark cluster and allocating them to

spark job

(30) Choosing a cluster manager for any Spark application depends on the goals of the application.

(31) The standalone cluster manager is the easiest one to use when developing a new Spark application.

(32) Cluster is used to deploy spark application and we don't need any 3rd party cluster manager.

(33) Spark Deployment Modes: Spark Standalone, Spark on Apache Mesos, Spark on Hadoop YARN, Amazon EC2, Kubernetes

(34) Running Spark application on YARN:

Spark preconfigured for YARN and it does not require any additional configuration to run.

YARN controls the resource management, scheduling, and security when we run Spark applications on it

It is possible to run an application in any mode, whether it is cluster mode (many nodes) or client mode(one single entity).

Cluster Deployment Mode:

(35) The spark driver runs inside an ApplicationMaster(AM) process which is managed by YARN

(36) Thus, the client can go away after initializing the application

(37) A single process in a YARN container is responsible for driving the application and requesting resources from YARN

---

(38) Spark Shell:

spark shell provides a simple way to learn the API, as well as a powerful tool to analyze data interactively.

spark comes with the three types of interactive shells:

PySpark with Python Support, Spark shell with Scala support, Spark R with R support

> (39) Spark Web UI:

Every sparkcontext launches a web UI

A web ui includes: jobs, stages, storage, environment, executors.

> (40) The "Jobs" tab in a web ui shows the status of all spark jobs in a spark application (i.e. a sparkcontext)

The Jobs tab consists of two pages: All Jobs, Details for Job

The tab displays stages per state, such as active, pending, completed, skipped or failed

> (41) The "Stages" tab is created exclusively when a SparkUI is initialized.

It shows the current state of all stages.

The Stages tab includes: AllStagesPage, StagePage, PoolPage

> (42) The "Storage" tab displays the information about RDDs

The summary page in the Storage tab shows the storage levels and partitions of all RDDs

It also shows the sizes and the executors used for all partitions in an RDD

The Storage tab includes the following pages: StoragePage, RDDPage

> (43) The "Environment" tab displays the values of different environment and configuration variables

It shows Java, Spark, and system properties

The Environment tab has: Parent SparkUI, AppStatusStore

> (44) The "Executors" tab displays the summary information about the executors that were created for the application

It displays memory and disk usage and task and shuffle information

The Storage Memory column in the Executors tab shows the amount of memory used and reserved for caching data for access
or near real time application.

> (45) PySpark Shell:

PySpark shell exposes the Spark programming model to python.

PySpark shell links the Python API to the Spark Core and initializes the SparkContext

PySpark requires Python to be available on the system PATH and uses it to run programs

> (46) Submitting a PySpark Job:

Many times our code depends on other projects or source codes, so we need to package them alongside our application to
distribute the code to a spark cluster

For this, we create an assembly jar containing our code and its dependencies

Both SBT and Maven have assembly plugins

When creating assembly jars, list Spark and Hadoop as provided dependencies

Once we have an assembled jar, we can call the bin/spark-submit script while passing our jar

The spark-submit script is used to launch application on the cluster

> (47) The following steps will help us to build our first application in spark using python:

- -- Create a document named PySparkjob.py on the local drive
- -- open PySparkjob.py in notepad and write: print("hello! this is pyspark") [source code]
- -- We can upload the PySparkjob.py file on our CloudLab's local storage if we are not using a VM and perform the following steps:

check current directory where we have the above file using ls

run: spark-submit --master yarn --deploy-mode client PySparkjob.py

it will show the output.

> (48) PySpark Job using Jupyter Notebooks

write the following code in jupyter notebook:

from pyspark import SparkConf
from pyspark import SparkContext

```
conf = SparkConf()
conf.setMaster('local')
conf.setAppName('spark-basic')
sc= SparkContext(conf=conf)

def mod(x):
import numpy as np
return (x,np.mod(x,2))

rdd = sc.parallelize(range(1000)).map(mod).take(10)
print(rdd)
```

- -- RDD stands for resilient distributed dataset. RDDs are the fundamental structure of spark. Every dataset in RDD
  is divided into various local partitions which is needed for different clusters.
- -- Spark application most of the times prefer java version 8. (check colab notebook file)

notebook file:

```
#creating spark session
import findspark
findspark.init()
from pyspark.sql import SparkSession
spark=SparkSession.builder.master("local[*]").getOrCreate()

# stopping the session

spark.stop()

# installing Pyspark

pip install pyspark

import pyspark

# creating a spark context

from pyspark import SparkConf
from pyspark import SparkContext
conf=SparkConf()
conf.setMaster('local')
conf.setAppName('spark-basic')
sc=SparkContext(conf=conf)

def mod(x):
import numpy as np
return (x,np.mod(x,2))

# creating an RDD
```

```
rdd = sc.parallelize(range(1000)).map(mod).take(10)
print(rdd)
```

```
# creating an RDD using List
```

```
values = [1,2,3,4,5]
rdd = sc.parallelize(values)
```

> (49) Spark RDDs:

RDDs are immutable and structured partitions of data

> (50) Spark RDDs supports:

- A. In-Memory Computation
- B. Lazy Evaluation
- C. Fault Tolerance
- D. Immutability
- E. Partitioning
- F. Persistence
- G. Coarse Grained Operation

> (51) RDDs are the main logical data unit in spark.

They are a distributed collection of objects, stored in memory or on disks of different machines of a cluster

A single RDD can be divided into multiple logical partitions so that these partitions can be stored and
processed on different machines of a cluster

RDDs in spark can be cached and used again for future transformations

They are lazily evaluated i.e. spark delays the RDD evaluation until it is really needed

We can perform transformations and actions on RDDs.

> (52) RDD Workflow:

- A. Many RDDs are clubbed into a single RDD.
- B. RDDs are arrays or structured data and this structured data is converted into DAG(Directed acyclic graph)
  so that data can be flow from one node to another in structured manner. This is the work of DAGScheduler.

- C. Once DAGScheduler finished its job where it assembled, ordered and stored it. Now, it passed it into Cluster Manger.
  and it pulls the data from all places and each operation is considered a task which is done by Task Scheduler and it
  passed the task to Executor
- D. Executor gives the required output.

> (53) The Existing Technologies:

Before Spark RDDs, large volumes of data were processed using in-disk computation, which is slower than
in-memory computation

In-disk computation uses memory capacity to process stored data

The existing distributed computing systems (viz. MapReduce) need to store data in some intermediate stable distributed
store, namely HDFS

It makes the overall computations of jobs slower as it involves multiple IO operations, replications and
serializations in the process

Besides spark, most of the data processing frameworks are less efficient to perform parallel processing.

> (54) How do RDDs solve the problem ?

- -- Spark RDDs have various capabilities to handle huge volumes of data by processing them in parallel over
  multiple logical partitions. We can create an RDD, anytime, but Spark RDDs are executed only when they are needed
  (lazy evaluation)
- -- Here are some more facts that explain why RDDs are better than the existing frameworks:
- A. Enhanced Distributed Computing:

Spark RDDs are useful for distributed computing i.e. processing data over multiple jobs

- B. Partitioning:

RDDs are partitioned (split into logical partitions) and distributed across nodes in a cluster

- C. Location Stickiness:

RDDs can define their placement preference (the location) to compute partitions.

> (55) Features of Spark RDDs:

- A. In-memory Computation:

The data inside an RDD can be stored in memory for as long as possible

- B. Immutable or Read-Only:

An RDD can't be changed once created and can only be transformed using transformations

- C. Lazy Evaluation:

The data inside an RDD is not available or transformed until an action is executed that triggers the execution

- D. Cacheable:

Cache stores the intermediate RDD results in memory only, i.e. the default storage of RDD cache is in memory
to make access fast.

- E. Parallel Data Processing:

RDDs can process data in parallel

- F. Typed:

RDD records have types e.g. Int, Long, etc gives better data readability to check long type data etc.

> (56) Creating RDDs in PySpark:

```
from pyspark import SparkConf
from pyspark import SparkContext

# creating a spark context

conf = SparkConf().setAppNmae('SparkRDD').setMaster('local')
sc = SparkContext(conf=conf)
```

- -- The "conf" object is the configuration for a Spark application
- -- We define the App Name and the Master URL in it
- -- "sc" is an object of SparkContext

> (57) Creating an RDD using a List:

```
values = [1,2,3,4,5]
rdd = sc.parallelize(values) # parallelize take collections like list etc.
```

`# printing the RDD values:`

rdd.take(5)

> (58) # Uploading a file to Google Colab:

from google.colab import files
uploaded = files.upload()

`# initializing an RDD using a text file:`

rdd = sc.textFile("Spark.txt")

`# printing the text from RDD`

rdd.collect()

> (59) RDD Persistence and Caching:

- -- Spark RDDs are lazily evaluated; thus when we wish to use the same RDD multiple times, it results in recomputing the RDD
- -- To avoid computing an RDD multiple times, we can ask Spark to persist the data
- -- In this case, the node that computes the RDD stores its partitions
- -- If a node has data persisting on it fails, Spark will recompute the lost partitions of the data when required
- -- RDDs comes with a method called unpersist() that lets us manually remove them from the cache

aba = sc.parallelize(range(1,10000,2))
aba.persist()

> (60) Persistence Level:

Level --> Space Used --> CPU Time --> In Memory --> On Disk --> Comments

1. MEMORY_ONLY --> High --> Low --> Yes --> No --> Stores an RDD as a deserialized Java object in JVM. If it does not fit in memory, some partitions will not be cached and will be recomputed when needed
2. MEMORY_ONLY_SER --> Low --> High --> Yes --> No --> Stores an RDD as a serialized Java object. It stores one byte array per partition
3. MEMORY_AND_DISK --> High --> Medium --> Some --> Some --> Stores an RDD as a deserialized Java object in JVM. If the full RDD does not fit in memory, the remaining partitions is stored

on the disk, instead of recomputing it

4. MEMORY_AND_DISK_SER --> Low --> High --> Some --> Some --> Spills to the disk if there is too much data to fit in memory and stores serialized representation in memory
5. DISK_ONLY --> Low --> High --> No --> Yes --> Stores the RDD only on disk.

> (61) Caching:

- -- It is very useful when data is accessed repeatedly, such as, querying a small 'hot' dataset or running an iterative algorithm
- -- Cache is fault-tolerant
- -- Let is revisit our example and mark our linesWithSpark dataset to be cached

```
textFile = sc.textFile("Spark.txt")
textFile.cache()
```

> (62) Operations on RDDs:

There are two types of data operations we can perform on an RDD, transformations and actions

- -- A transformation will return a new RDD as RDDs are generally immutable
- -- An action will return a value

> (63) Transformations are lazy operations on an RDD that create one or more new RDDs.

RDD transformations return a pointer to the new RDD and allow us to create dependencies between RDDs.
Each RDD in a dependency chain (a string of dependencies) has a function for calculating its data and
a pointer (dependency) to its parent RDD.

spark is lazy, so nothing will be executed unless we call some transformation or action that will trigger the
job creation and execution.

Therefore, RDD transformation is not a set of data but a step in a program (might be the only step) telling Spark how to get
data and what to do with it.

> (64) Given below is a list of RDD Transformations:

map
flatMap
filter
mapPartitions
mapPartitionsWithIndex
sample
union
intersection
distinct
groupBy
keyBy
Zip
zipwithIndex
Coalesce
Repartition
sortBy

> (65) Map:

it passes each element through a function.

```
x=sc.parallelize(["spark","rdd","example","sample","example"])
y=x.map(lambda x: (x,1))
y.collect()
```

> (66) flatMap:

It is like map, but here each input item can be mapped to 0 or more output items (so, a function should
return a sequence rather than a single item)

```
rdd = sc.parallelize([2,3,4])
sorted(rdd.flatMap(lambda x: range(1,x)).collect())
```

> (67) filter:

Returns a collection of elements on the basis of the condition provided in the function

```
rdd = sc.parallelize([1,2,3,4,5])
rdd.filter(lambda x: x%2 ==0).collect()
```

o/p: [2,4]

> (68) Sample:

- -- it samples a fraction of the data with or without replacement, using a given random number generator seed.
- -- We can add following parameters to sample method:
- A. withReplacement: Elements can be sampled multiple times (replaced when sampled out)
- B. fraction: Makes the size of the sample as a fraction of the RDD's size without replacement
- C. seed: A number as a seed for the random number generator

```
parallel=sc.parallelize(range(9))
parallel.sample(True,.2).count()
```

o/p: 1

```
parallel.sample(False,1).collect()
```

> (69) Union:

It returns the union of two RDDs after concatenating their elements.

```
parallel = sc.parallelize(range(1,9))
par = sc.parallelize(range(5,15))
parallel.union(par).collect()
```

o/p: [1,23,4,...,14]

> (70) Intersection:

similar to union, but returns the intersection of two RDDs

```
parallel = sc.parallelize(range(1,9))
par = sc.parallelize(range(5,15))
parallel.intersection(par).collect()
```

o/p: [6,8,5,7]

> (71) Distinct:

Returns a new RDD with distinct elements within the source data

```
parallel = sc.parallelize(range(1,9))
par = sc.parallelize(range(5,15))
parallel.union(par).distinct().collect()
```

o/p: [2,4,6,...,13]

> (72) sortBy:

It returns the RDD sorted by the given key function.

```
y = sc.parallelize([5,7,1,3,2,1])
y.sortBy(lambda c: c, True).collect()
```

o/p: [1,1,2,3,5,7]

```
z=sc.parallelize([("H",10),("A",26),("Z",1),("L",5)])
z.sortBy(lambda c:c,False).collect()
```

o/p: [('Z',1),('L',5),('H',10),('A',26)]

> (73) MapPartitions:

can be used as an alternative to map() and foreach()

mapPartition() is called once for each partition unlike map() and foreach(), which are called for each element in the RDD

```
rdd = sc.parallelize([1,2,3,4],2)
def f(iterator): yield sum(iterator)
rdd.mapPartitions(f).collect()
```

o/p: [3,7]

> (74) rdd = sc.parallelize([1,2,3,4],4)
> def f(splitIndex,iterator): yield splitIndex
> rdd.mapPartitionWithIndex(f).sum()

o/p: 6

> (75) groupBy:

Returns a new RDD by grouping objects in the existing RDD using the given grouping key

```
rdd = sc. parallelize([1,1,2,3,5,8])
result= rdd.groupBy(lambda x: x%2).collect()
sorted([(x,sorted(y)) for (x,y) in result])
```

o/p: [(0,[2,8]),(1,[1,1,3,5])]

> (76) keyBy:

It returns a new RDD by changing the key of the RDD element using the given key object

```
x = sc.parallelize(range(0,3)).keyBy(lambda x: x*x)
y = sc.parallelize(zip(range(0,5),range(0,5)))
[(x,list(map(list,y))) for x,y in sorted(x.cogroup(y).collect())]
```

> (77) zip:

```
x = sc.parallelize(range(0,5))
y = sc.parallelize(range(1000,1005))
x.zip(y).collect()
```

o/p: [(0,1000),(1,1001),(2,1002),(3,1003),(4,1004)]

```
sc.parallelize(["a","b","c","d"],3).zipWithIndex().collect()
```

o/p: [('a',0),('b',1),('c',2),('d',3)]

> (78) zip joins two RDDs by combining the ith part of either partition with each other.
> Returns an RDD formed from this list and another iterable collection by combining the corresponding elements in pairs.
> If one of the two collections is longer than the other, its remaining elements are ignored.

> (79) repartition:

Used to either increase or decrease the number of partitions in an RDD.

Does a full shuffle and creates new partitions with the data that are distributed evenly.

```
rdd = sc.parallelize([1,2,3,4,5,6,7],4)
sorted(rdd.glom().collect())
```

o/p: [[1],[2,3],[4,5],[6,7]]

```
len(rdd.repartition(2).glom().collect())
```

o/p: 2

> (80) Coalesce:

This method is used to reduce the number of partitions in an RDD.

```
sc.parallelize([1,2,3,4,5],3).glom().collect()
```

o/p: [[1],[2,3],[4,5]]

```
sc.parallelize([1,2,3,4,5],3).Coalesce(2).glom().collect()
```

o/p: [[1],[2,3,4,5]]

> (81) Difference Between Coalesce() and Repartition():

- A. Coalesce() uses the existing partitions to minimize the amount of data that's shuffled
  Repartition() creates new partitions and does a full shuffle.
- B. Coalesce results in partitions with different amounts of data (at times with much different sizes)
  Repartition results in roughly equal-sized partitions
- C. Coalesce() is faster and Repartition() is not so faster

> (82) Operations on RDDs: Actions

- -- Unlike transformations that produce RDDs, action functions produce a value back to the spark driver program.
- -- Actions may trigger a previously constructed, lazy RDD to be evaluated

Given below is a list of commonly used RDD actions:

Reduce (func)
first
takeOrdered
take
count
collect
collectMap
saveAsTextFile
foreachPartition
Foreach
Max
Min
Sum
Mean
Variance
stdev

> (83) reduce:

Aggregate elements of a dataset through a function.

from operator import add
sc.parallelize([1,2,3,4,5]).reduce(add)

o/p: 15

sc.parallelize((2 for _ in range(10))).map(lambda x: 1).cache().reduce(add)

o/p: 10

> (84) first:

returns the first element in an RDD:

sc.parallelize([2,3,4]).first()

o/p: 2

> (85) takeOrdered:

Returns an array with the given number of ordered values in an RDD

```
nums = sc.parallelize([1,5,3,9,4,0,2])
nums.takeOrdered(5)
```

o/p: [0,1,2,3,4]

> (86) take:

Returns an array as the specified number in the take method.

nums = sc.parallelize([1,5,3,9,4,0,2])

nums.take(5)

o/p: [1,5,3,9,4]

> (87) count:

It returns a long value indicating the number of elements present in an RDD.

```
nums = sc.parallelize([1,5,3,9,4,0,2])
nums.count()
```

o/p: 7

> (88) Collect:

Returns the elements of the dataset as an array back to the driver program

Should be used wisely as all worker nodes return the data to the driver node

If the dataset is huge in size then this may result in an OutOfMemoryError

```
c = sc.parallelize(["Gnu","Cat","Rat","Dog","Gnu","Rat"],2)
c.collect()
```

o/p: ['Gnu','Cat','Rat','Dog','Gnu','Rat']

```
c = sc.parallelize(["Gnu","Cat","Rat","Dog","Gnu","Rat"],2)
c.distinct.collect()
```

o/p: ['Gnu','Cat','Rat','Dog']

> (89) "SaveAsTextfile" - filepath:

Writes the entire RDD's dataset as a text file on the path specified in the local filesystem or HDFS:

```
a=sc.parallelize(range(1,10000),3)
a.saveAsTextFile("/usr/bin/mydata_a1")
x=sc.parallelize([1,2,3],3)
x.saveAsTextFile("/usr/bin/sample1.txt")
```

> (90) foreach:

Passes each element in an RDD through the specified function.

```
def f(x): print(x)
sc.parallelize([1,2,3,4,5]).foreach(f)
```

> (91) foreach -- Partition:

```
def f(iterator):
for x in iterator:
print(x)
sc.parallelize([1,2,3,4,5]).foreachPartition(f)
```

> (92) max,min,sum,mean,variance and stdev:

```
numbers = sc.parallelize(range(1,100))
numbers.sum()
numbers.min()
numbers.variance()
numbers.max()
numbers.mean()
numbers.stdev()
```

> (93) RDD Functions:

cache() --> caches an RDD to use without computing again
collect() --> returns an array of all elements in an RDD
countByValue() --> returns a mao with the number of times each value occurs
distinct() --> returns an RDD containing only distinct elements
filter() --> returns an RDD containing only those elements that match wise the function f
foreach() --> Applies the function f to each elements of an RDD
persist() --> sets an RDD with the default storage level (MEMORY_ONLY); sets the storage level that caches the RDD TO BE STORED AFTER IT IS COMputed (different storage level is there in StorageLevel)
sample() --> returns an RDD of that fraction
toDebugString() --> Returns a handy function that outputs the recursive steps of an RDD
count() --> Returns the number of elements in an RDD
unpersist() --> Removes all the persistent blocks of an RDD from the memory/disk
union() --> returns an RDD containing elements of 2 RDDs; duplicates are not removed

> (94) countByValue:

a=sc.parallelize([1,2,3,4,5,6,7,8,2,3,3,3,1,1,1])
a.countByValue()

o/p: defaultdict(int, {1:4,2:3,3:4,4:2,5:1,6:1,7:1,8:1})

> (95) toDebugString:

a=sc.parallelize(range(1,19),3)
b=sc.parallelize(range(1,13),3)
c=a.subtract(b)
c.toDebugString()

o/p: b'(6) PythonRDD[165] at RDD at PythonRDD.scala:53 [] \n

(to find where RDD is stored)

> (96) Creating Paired RDDs (working with key-value pair):

- -- There are a number of ways to get paired RDDs in spark.
- -- There are many formats that directly return paired RDDs for their key-value data; in other cases, we have regular RDDs that need to be turned into paired RDDs
- -- We can do this by running a map() function that returns key-value pairs

> (97) rdd = sc.parallelize([("a1","b1","c1","d1","e1"),("a2","b2","c2","d2","e2")])
> result = rdd.map(lambda x: (x[0],list(x[1:])))

> result.collect()

o/p: [('a1',['b1','c1','d1','e1']),('a2',['b2','c2','d2','e2'])]

> (98) Transformations on Paired RDDs:

- -- Transformations on one paired RDD.
- -- E.g.: RDD = {(1,2),(3,4),(3,6)}

Functions --> Purpose --> Example --> Result

reduceByKey(func) --> Combines values with the same key --> rdd.reduceByKey(add) --> [(1,2),(3,10)]
groupByKey() ==> Groups values with the same key --> rdd.groupByKey() --> {(1,[2]),(3,[4,6])}
mapValues(func) --> Applies a function to each value of a paired RDD without changing the key --> rdd.mapValues(lambda x: x+1) --> {(1,3),(3,5),(3,7)}
flatMapValues(func) --> applies a function that returns an iterator to each value of a paired RDD and for each element returned, produces a key-value entry with the old key often used for --> rdd.flatMapValues(lambda x: range(x,5)) --> {(1,2),(1,3),(1,4),(1,5),(3,4),(3,5)}
keys() --> Returns an RDD of just the keys --> rdd.keys() --> {1,3,3}
sortByKey() --> Between an RDD sorted by the key --> rdd.sortByKey() --> {(1,2),(3,4),(3,6)}
subtractByKey() --> Removes elements with the key present in the other RDD --> rdd.subtractByKey(other) ==> {(1,2)}
join() --> Performs an inner join between both RDDs --> rdd.join(other) --> {{3,{4,9}},{3,{6,9}}}
rightOuterJoin() --> Performs a join between 2 RDDs where the key must be present in the first RDD --> rdd.rightOuterJoin(other) --> {{3,{Some{4},9}},{3,{Some{6},9}}}

leftOuterJOIN()
cogroup()

> (99) RDD Lineage:

RDD Lineage is a graph of all parent RDDs of an RDD.
It is built by applying transformations to the RDD and creating a logical execution plan.
Consider the following series of transformations:

rdd.toDebugString # to print rdd image

An RDD lineage graph is a graph of transformations that need to be executed after an action has been called.
We can create an RDD lineage graph using the RDD.toDebugString method.

> (100) WordCount using RDD concepts:

- -- it returns the frequency of every word.

```
rdd = sc.textFile("PySpark.txt")
nonempty_lines = rdd.filter(lambda x: len(x)>0)
words = nonempty_lines.flatMap(lambda x: x.split(' '))
wordcount = words.map(lambda x: (x,1)).reduceByKey(lambda x,y:x+y).map(lambda x:(x[1],x[0])).sortByKey(False)

for word in wordcount.collect():
print(word)
```

> (101) RDD Partitioning:

- -- A partition is a logical chunk of a large distributed dataset
- -- Spark manages data using partitions that help parallelize distributed data processing with minimal network traffic
  for sending data between executors.
- -- By default, Spark tries to read data into an RDD from the nodes that are close to it.
- -- Since Spark usually accesses distributed partitioned data, to optimize transformations, it creates partitions
  that can hold the data chunks
- -- RDDs get partitioned automatically without a programmer's intervention
- -- However there are times when we would like to adjust the size and number of partitions or the partitioning scheme
- -- We can use the def getPartition: Array[Partition] method on an RDD to know the number of partition in the RDD

> (102) RDD Partitioning Types:

- A. Hash Partitioning
- B. Range Partitioning

Hash Partitioning is a partitioning is a partitioning technique where a hash key is used to distribute elements evenly across different partitions.

Some Spark RDDs have keys that follow a particular order; for such RDDs, range partitioning is an efficient partitioning technique/
In the range partitioning method, tuples having keys within the same range will appear on the same machine
Keys in a RangePrtitioner are partitioned based on the set of sorted range of keys and ordering of keys.

- -- Customizing partitioning is possible only on paired RDDs.